Title: Design and Implementation of an Agentic AI-Based Intelligent System for Employee Attrition Prediction

---

Student Name:

Student ID:

Table of Contents

1. Application and Workflow Selection 4

1.1 Introduction and Motivation of the Selected Application.....4

1.2 Explanation of the Selected Workflow with Diagram.....4

1.3 Objectives of the Selected Intelligent System.....5

1.4 Impact of the Selected Project if Developed.....6

2. Selection of AI Technologies 6

2.1 Identification of Similar Systems 6

2.2 Innovative and Original Approach 6

2.3 Justification for AI Methods and Technologies.....7

3. Simulations and Development Setup 7

3.1 Simulation Environment and Setup Parameters.....7

3.2 Agent-wise Task Distribution, Functionality, and Datasets 8

3.3 Encoding, Training, and Testing Process 9

4. Use Case Explanation with Evidence.....9

4.1 Step-by-Step Workflow Execution 9

4.2 Developed User Interface and External Interaction 20

5. Conclusions.....21

6. References.....23

List of Figures

Figure 1 Workflow Diagram 5
Figure 2 Install Kaggel (Development Setup) 8
Figure 3 Agent 1 Implementations 10
Figure 4 Agent 1 Output 10
Figure 5 Agent 2 Implementations 1 11
Figure 6 Agent 2 Implementations 2 12
Figure 7 Agent 2 Output 12
Figure 8 Agent 2 Attrition Class Distribution After Encoding 13
Figure 9 Agent 2 Feature Variance After Scaling 13
Figure 10 Agent 3 Implementations 14
Figure 11 Agent 3 Output 14
Figure 12 Agent 4 Implementations 15
Figure 13 Agent 4 Output 15
Figure 14 Agent 4 Decision Tree - Confusion Matrix 16
Figure 15 Agent 4 Random Forest Graph 16
Figure 16 Agent 4 Random Forest - Confusion Matrix 17
Figure 17 Agent 4 Logistic Regression Graph 17
Figure 18 Agent 4 Logistic Regression - Confusion Matrix 18
Figure 19 Agent 4 ROC Curve Comparison Graph 18
Figure 20 Agent 5 Implementations 1 19
Figure 21 Agent 5 Implementations 2 19
Figure 22 Agent 5 Output 19
Figure 23 Agent 6 Implementations 20
Figure 24 Agent 6 Output 20

1. Application and Workflow Selection

1.1 Introduction and Motivation of the Selected Application

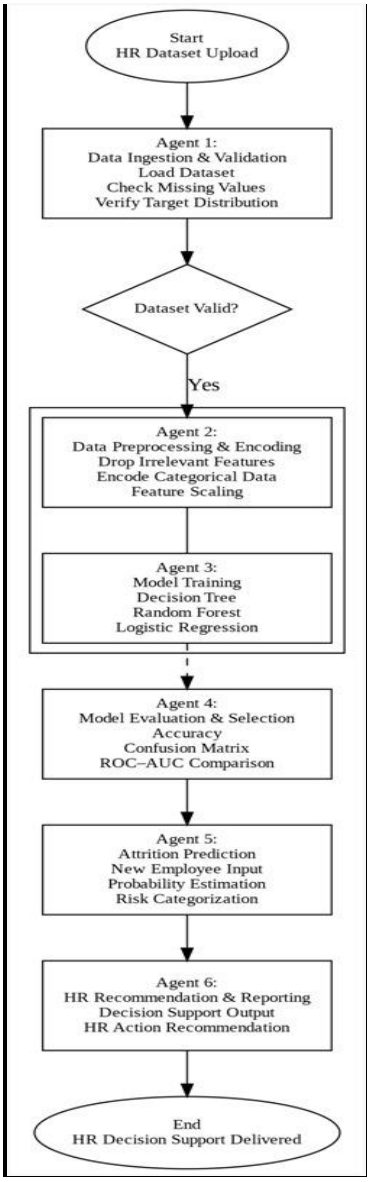
The field of human resource management has been more data-based, with organizations aiming at enhancing employee retention and productivity as well as enhancing employee stability. Employee attrition is among the most pressing issues of HR departments that lead to higher expenses of recruitment, loss of company knowledge, and decreased operation performance. The common approaches to determining attrition risks are based on administrative methods whereby personnel are judged manually or through the exit interview method and past trends, which prove to be reactive and not proactive. In order to overcome this difficulty, this project aims at the conception of an intelligent employee attrition predicting system based on the Agentic AI approach design (Sapkota et al., 2025). The system uses machine learning models to offer insights into the probability of an employee attraction depending on demographic, professional, and performance-related factors. It will simulate an HR analytics pipeline in the real world by adding autonomous agents that operate in a sequence to allow one to detect at-risk employees early (Faruk, 2025). This creative approach can tremendously improve the retention and HR planning strategies.

1.2 Explanation of the Selected Workflow with Diagram

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# Install Kaggle
!pip install -q kaggle

KAGGLE_USERNAME = "your_kaggle_username"
KAGGLE_KEY = "your_kaggle_api_key"

import os

# Set environment variables
os.environ["KAGGLE_USERNAME"] = KAGGLE_USERNAME
os.environ["KAGGLE_KEY"] = KAGGLE_KEY

# Create directory for dataset
os.makedirs("ibm_hr_attrition_dataset", exist_ok=True)

# Download the dataset
!kaggle datasets download -d pavansubhasht/ibm-hr-analytics-attrition-dataset -p ibm_hr_attrition_dataset

# Unzip the dataset
!unzip -o ibm_hr_attrition_dataset/ibm-hr-analytics-attrition-dataset.zip -d ibm_hr_attrition_dataset

# List downloaded files
os.listdir("ibm_hr_attrition_dataset")
```

```
Dataset URL: https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset
License(s): DbCL-1.0
Downloading ibm-hr-analytics-attrition-dataset.zip to ibm_hr_attrition_dataset
  0% 0.00/50.1k [00:00<?, ?B/s]
100% 50.1k/50.1k [00:00<00:00, 157MB/s]
Archive:  ibm_hr_attrition_dataset/ibm-hr-analytics-attrition-dataset.zip
  inflating: ibm_hr_attrition_dataset/WA_Fn-UseC_-HR-Employee-Attrition.csv
['WA_Fn-UseC_-HR-Employee-Attrition.csv',
 'ibm-hr-analytics-attrition-dataset.zip']
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

AGENT 1: Data Ingestion & Validation Agent

```python
import pandas as pd

# Load the dataset
file_path = "ibm_hr_attrition_dataset/WA_Fn-UseC_-HR-Employee-Attrition.csv"
df = pd.read_csv(file_path)

# Display basic dataset information
print("Dataset Shape (Rows, Columns):")
print(df.shape)

print("\nDataset Columns:")
print(df.columns.tolist())

print("\nFirst 5 Rows of Dataset:")
display(df.head())

# Check for missing values
print("\nMissing Values per Column:")
print(df.isnull().sum())

# Check target variable distribution
print("\nAttrition Value Counts:")
print(df['Attrition'].value_counts())

print("\nAgent 1 Status: Dataset successfully ingested and validated.")
```

[redacted]

```
Dataset Shape (Rows, Columns):
(1470, 35)

Dataset Columns:
['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department', 'DistanceFromHome', 'Education', 'EducationField', 'EmployeeCount',

First 5 Rows of Dataset:
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	1
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	2
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	4
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	5
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	7

5 rows × 35 columns

```
Missing Values per Column:
Age                       0
Attrition                 0
BusinessTravel            0
DailyRate                 0
Department                0
DistanceFromHome          0
Education                 0
EducationField            0
EmployeeCount             0
EmployeeNumber            0
```

[redacted]

[redacted]

[redacted]

AGENT 2: Data Preprocessing & Encoding Agent

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Create a working copy
df_processed = df.copy()


# Step 1: Drop Irrelevant Columns

drop_columns = ['EmployeeNumber', 'EmployeeCount', 'StandardHours', 'Over18']
df_processed.drop(columns=drop_columns, inplace=True)

print("Dropped Columns:")
print(drop_columns)


# Step 2: Encode Target Variable

le = LabelEncoder()
df_processed['Attrition'] = le.fit_transform(df_processed['Attrition'])

print("\nTarget Encoding:")
print(dict(zip(le.classes_, le.transform(le.classes_))))


# Step 3: Encode Categorical Features

categorical_columns = df_processed.select_dtypes(include=['object']).columns.tolist()

for col in categorical_columns:
    df_processed[col] = le.fit_transform(df_processed[col])

print("\nEncoded Categorical Columns:")
print(categorical_columns)
```

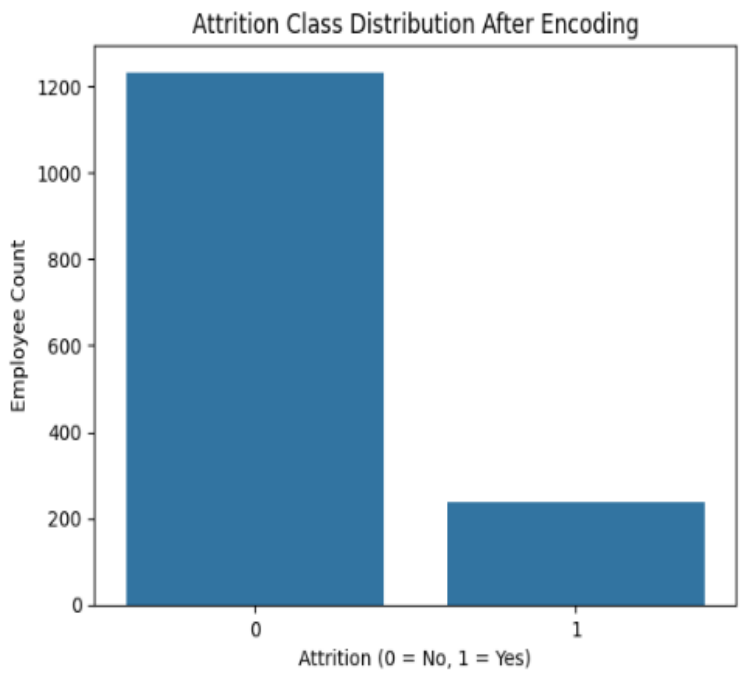
[redacted]

```python
# Step 4: Feature-Target Separation

X = df_processed.drop('Attrition', axis=1)
y = df_processed['Attrition']

print("\nFeature Matrix Shape:", X.shape)
print("Target Vector Shape:", y.shape)


# Step 5: Feature Scaling

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\nFeature Scaling Applied using StandardScaler")


# VISUALIZATIONS


# Attrition distribution after encoding
plt.figure()
sns.countplot(x=y)
plt.title("Attrition Class Distribution After Encoding")
plt.xlabel("Attrition (0 = No, 1 = Yes)")
plt.ylabel("Employee Count")
plt.show()

# Feature variance visualization (first 10 features)
plt.figure()
pd.DataFrame(X_scaled[:, :10], columns=X.columns[:10]).var().plot(kind='bar')
plt.title("Feature Variance After Scaling (Sample Features)")
plt.ylabel("Variance")
plt.show()

print("\nAgent 2 Status: Data preprocessing, encoding, and visualization completed successfully.")
```

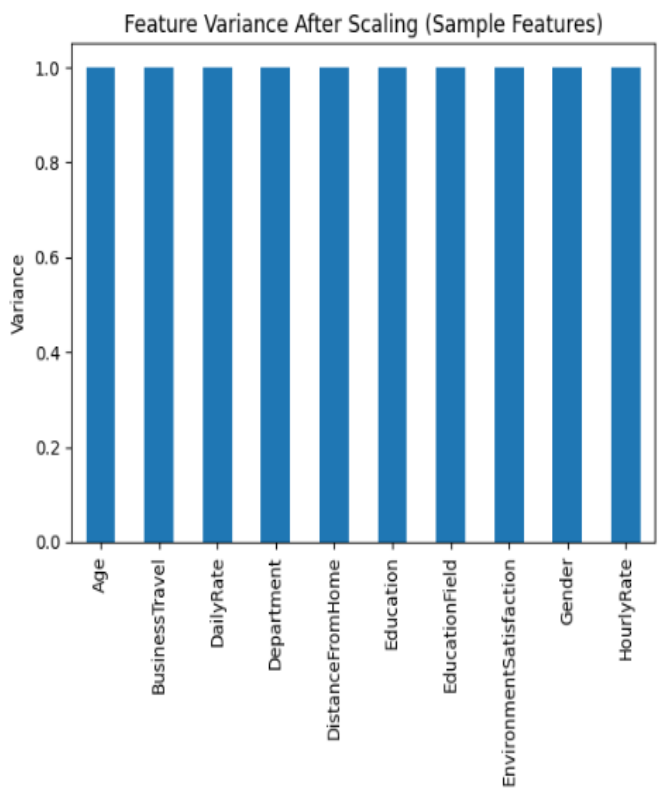
[redacted]

```
Dropped Columns:
['EmployeeNumber', 'EmployeeCount', 'StandardHours', 'Over18']

Target Encoding:
{'No': np.int64(0), 'Yes': np.int64(1)}

Encoded Categorical Columns:
['BusinessTravel', 'Department', 'EducationField', 'Gender', 'JobRole', 'MaritalStatus', 'OverTime']

Feature Matrix Shape: (1470, 30)
Target Vector Shape: (1470,)

Feature Scaling Applied using StandardScaler
```

[redacted]



[redacted]




```
Agent 2 Status: Data preprocessing, encoding, and visualization completed successfully.
```

[redacted]

[redacted]

AGENT 3: Model Training and Learning Agent

```python
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

# Train-test split (80-20)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

print("Training Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

# Initialize models
dt_model = DecisionTreeClassifier(random_state=42)
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
lr_model = LogisticRegression(max_iter=1000)

# Train models
dt_model.fit(X_train, y_train)
rf_model.fit(X_train, y_train)
lr_model.fit(X_train, y_train)

print("\nModels Trained Successfully:")
print("- Decision Tree")
print("- Random Forest")
print("- Logistic Regression")

print("\nAgent 3 Status: Model training completed.")
```

[redacted]

```
Training Samples: 1176
Testing Samples: 294

Models Trained Successfully:
- Decision Tree
- Random Forest
- Logistic Regression

Agent 3 Status: Model training completed.
```

[redacted]

[redacted]

[redacted]

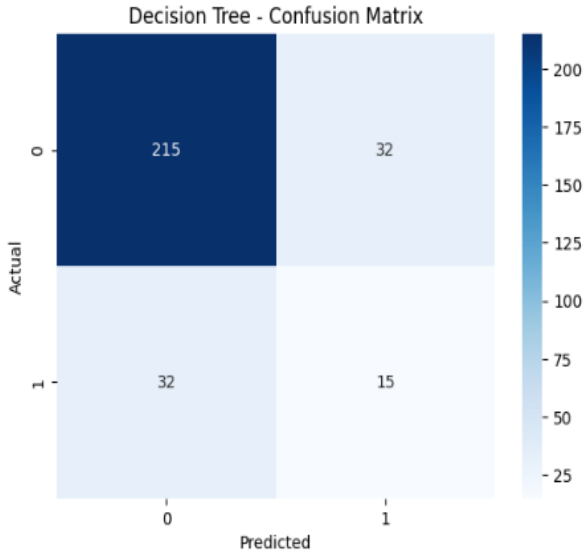
AGENT 4: Model Evaluation and Selection Agent

```python
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, roc_curve, auc
)
import matplotlib.pyplot as plt
import seaborn as sns

models = {
    "Decision Tree": dt_model,
    "Random Forest": rf_model,
    "Logistic Regression": lr_model
}

plt.figure()

for name, model in models.items():
    # Predictions
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    # Metrics
    acc = accuracy_score(y_test, y_pred)
    print(f"\n{name} Accuracy: {acc:.4f}")
    print(classification_report(y_test, y_pred))

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    plt.figure()
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f"{name} - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

    # ROC Curve
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    roc_auc = auc(fpr, tpr)

    plt.plot(fpr, tpr, label=f"{name} (AUC = {roc_auc:.2f})")

plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.show()

print("\nAgent 4 Status: Model evaluation and comparison completed.")
```

[redacted]

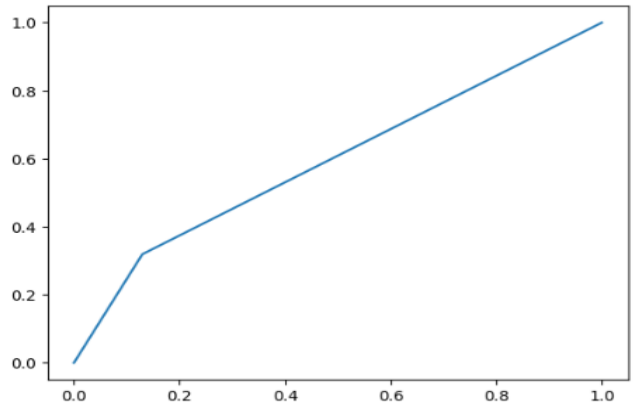
```
...
    Decision Tree Accuracy: 0.7823
                  precision    recall  f1-score   support

               0       0.87      0.87      0.87       247
               1       0.32      0.32      0.32        47

        accuracy                           0.78       294
       macro avg       0.59      0.59      0.59       294
    weighted avg       0.78      0.78      0.78       294

    <Figure size 640x480 with 0 Axes>
```

[redacted]

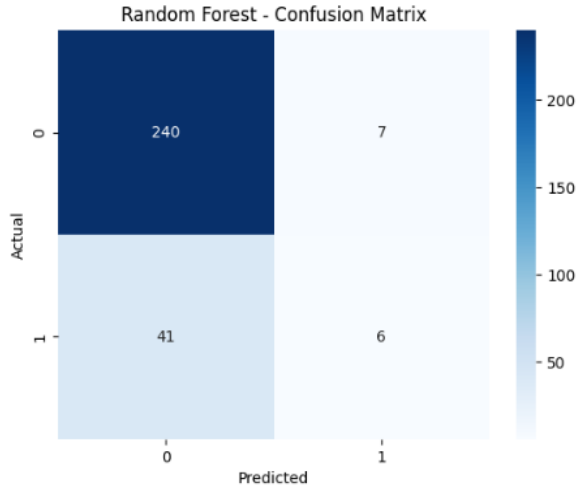


[redacted]

```
Random Forest Accuracy: 0.8367
              precision    recall  f1-score   support

           0       0.85      0.97      0.91       247
           1       0.46      0.13      0.20        47

    accuracy                           0.84       294
   macro avg       0.66      0.55      0.55       294
weighted avg       0.79      0.84      0.80       294
```

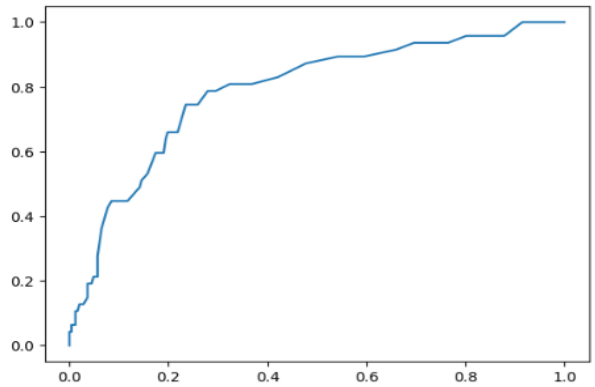


[redacted]

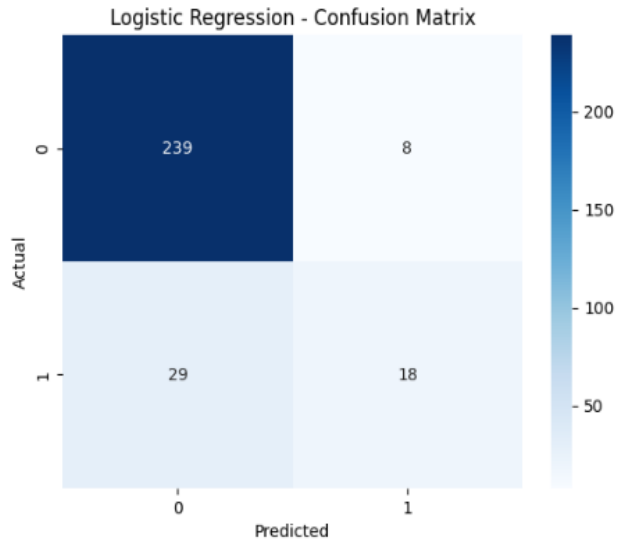


[redacted]

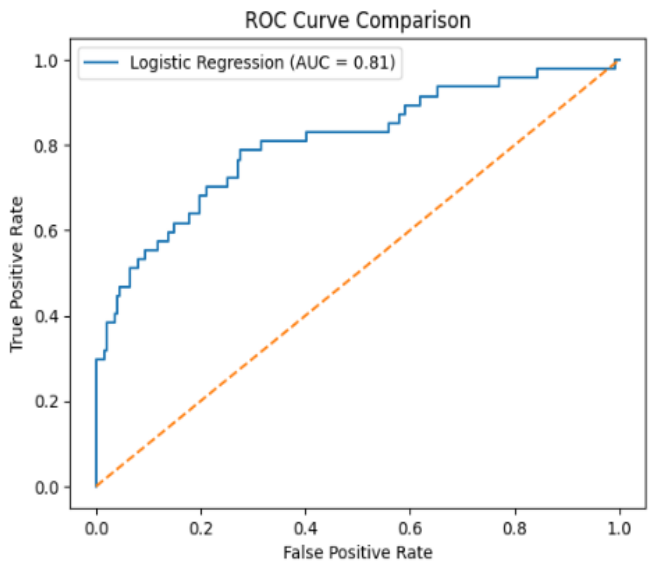
```
Logistic Regression Accuracy: 0.8741
              precision    recall  f1-score   support

           0       0.89      0.97      0.93       247
           1       0.69      0.38      0.49        47

    accuracy                           0.87       294
   macro avg       0.79      0.68      0.71       294
weighted avg       0.86      0.87      0.86       294
```



[redacted]



[redacted]




```
Agent 4 Status: Model evaluation and comparison completed.
```

[redacted]

[redacted]

[redacted]

AGENT 5: Attrition Prediction and User Interaction Agent

```python
import numpy as np

# Example new employee data (HR input)
# Order MUST match X.columns
new_employee = {
    'Age': 35,
    'BusinessTravel': 1,
    'DailyRate': 800,
    'Department': 1,
    'DistanceFromHome': 5,
    'Education': 3,
    'EducationField': 2,
    'EnvironmentSatisfaction': 3,
    'Gender': 1,
    'HourlyRate': 70,
    'JobInvolvement': 3,
    'JobLevel': 2,
    'JobRole': 5,
    'JobSatisfaction': 3,
    'MaritalStatus': 1,
    'MonthlyIncome': 6000,
    'MonthlyRate': 15000,
    'NumCompaniesWorked': 2,
    'OverTime': 1,
    'PercentSalaryHike': 12,
    'PerformanceRating': 3,
    'RelationshipSatisfaction': 3,
    'StockOptionLevel': 1,
    'TotalWorkingYears': 10,
    'TrainingTimesLastYear': 3,
    'WorkLifeBalance': 3,
    'YearsAtCompany': 5,
    'YearsInCurrentRole': 3,
    'YearsSinceLastPromotion': 1,
    'YearsWithCurrManager': 4
}
```

[redacted]

```python
# Convert input to DataFrame
new_df = pd.DataFrame([new_employee])

# Scale input using existing scaler
new_scaled = scaler.transform(new_df)

# Prediction
prediction = lr_model.predict(new_scaled)[0]
probability = lr_model.predict_proba(new_scaled)[0][1]

# Risk interpretation
if probability < 0.30:
    risk = "Low Risk"
elif probability < 0.60:
    risk = "Medium Risk"
else:
    risk = "High Risk"

print("Predicted Attrition:", "Yes" if prediction == 1 else "No")
print(f"Attrition Probability: {probability:.2f}")
print("Risk Category:", risk)

print("\nAgent 5 Status: Attrition prediction generated successfully.")
```

[redacted]

```
Predicted Attrition: No
Attrition Probability: 0.11
Risk Category: Low Risk

Agent 5 Status: Attrition prediction generated successfully.
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

AGENT 6: HR Recommendation and Reporting Agent

```python
def hr_recommendation(risk_level, probability):
    if risk_level == "High Risk":
        return (
            "High attrition risk detected. It is recommended to initiate immediate "
            "retention strategies, including salary revision, role clarity discussions, "
            "career development planning, and workload reassessment."
        )
    elif risk_level == "Medium Risk":
        return (
            "Moderate attrition risk identified. HR may consider targeted engagement "
            "initiatives, mentoring programs, and periodic performance feedback sessions."
        )
    else:
        return (
            "Low attrition risk observed. No immediate intervention required. "
            "Continue standard employee engagement and performance monitoring practices."
        )

# Generate HR recommendation
recommendation_text = hr_recommendation(risk, probability)

print("===== HR DECISION SUPPORT REPORT =====")
print("Predicted Attrition:", "Yes" if prediction == 1 else "No")
print(f"Attrition Probability: {probability:.2f}")
print("Risk Level:", risk)
print("\nRecommended HR Action:")
print(recommendation_text)

print("\nAgent 6 Status: HR recommendation and reporting completed successfully.")
```

[redacted]

```
===== HR DECISION SUPPORT REPORT =====
Predicted Attrition: No
Attrition Probability: 0.11
Risk Level: Low Risk

Recommended HR Action:
Low attrition risk observed. No immediate intervention required. Continue standard employee engagement and performance monitoring practices.

Agent 6 Status: HR recommendation and reporting completed successfully.
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]